# NSF SAFE-OSE: SECURING THE SCIENTIFIC PYTHON ECOSYSTEM

K. JARROD MILLMAN AND STÉFAN J. VAN DER WALT
UNIVERSITY OF CALIFORNIA, BERKELEY

NSF Safe-OSE: Securing the Scientific Python Ecosystem

## Project Summary


### Overview

This project aims to address critical safety, security, and privacy vulnerabilities in the Scientific Python Open Source Ecosystem (SP-OSE), a cornerstone of modern scientific computing, data analysis, and machine learning. The SP-OSE enables cutting-edge research across disciplines and is crucial for education, economic growth, and national security. We will leverage the Scientific Python Ecosystem Coordination (SPEC) process to develop and disseminate ecosystem-wide security standards and best practices. Our comprehensive approach addresses supply chain attacks, enhances access controls, and mitigates insider threats through community-supported initiatives and tooling.

### Intellectual Merit

The project's intellectual merit lies in its pioneering approach to securing a complex, decentralized open-source ecosystem. Through a community-backed coordination process, we will design and disseminate security mechanisms for a large, interconnected ecosystem of independently developed projects. The creation of a developer reputation system, a federated vulnerability management system, and automated anomaly detection represents a significant advancement in open-source security practices. Our approach combines technical expertise with community engagement experience, ensuring that security measures are both effective and widely adopted. The project will generate new knowledge about securing large-scale, collaborative scientific computing environments, which can be applied to other open-source ecosystems.

### Broader Impacts

By securing the SP-OSE, we will enhance the integrity and reliability of scientific research across numerous domains, including physics, astronomy, geospatial science, and neuroscience. This will lead to more trustworthy scientific outcomes and accelerate the pace of discovery. The project will strengthen national security by protecting critical infrastructure and sensitive data that rely on scientific Python tools. It will contribute to economic growth and innovation by securing the foundation of many data science and machine learning applications used in industry. By involving the community in the development of security measures, we will foster a culture of security awareness among open-source contributors and users. The project will set new standards for open-source security, benefiting not only the scientific Python ecosystem but also serving as a model for other open-source communities. Crucially, this work will safeguard the reputation of the SP-OSE as the de facto platform for scientific computing and data analysis, ensuring the continued vitality and resilience of this essential global research infrastructure. By proactively addressing security threats, we will preserve the ecosystem's role as a catalyst for innovation and a trusted foundation for scientific and technological advancement.



Keywords: Scientific Computing; Data Science; Machine Learning; Open Source Software; Research Infrastructure

NSF Safe-OSE: Securing the Scientific Python Ecosystem

## Project Description

1. **Introduction.** With its vast array of open-source tools, frameworks, and libraries, the **Scientific Python Open Source Ecosystem (SP-OSE)** [52, 60] has become an indispensable pillar of modern scientific computing, data analysis, and machine learning. The ecosystem supports the entire computational science lifecycle, from data gathering, to analysis, to publication and dissemination, and varies in generality from fundamental data structures, to general scientific algorithms, to domain-specific routines. Together with the *Jupyter Open Source Ecosystem (Jupyter-OSE)* [40], it enables cutting-edge research across disciplines, from astrophysics [4, 63, 12] to seismology [6, 50, 41]. Python's recent ascension to the #1 programming language on GitHub [25], surpassing JavaScript, underscores the critical importance and widespread use of these ecosystems. That surge in popularity also reflects the increasing centrality of data-driven approaches in science, education, industry, and defense, making the security and reliability of these ecosystems paramount to national and global scientific and technological progress.

2. **Current Status.** The SP-OSE is a mature, vibrant open-source ecosystem of independent, interconnected projects used by researchers across diverse scientific disciplines to perform complex analyses, simulations, and visualizations. The SP-OSE brings together people from around the world who work together to solve new scientific challenges. It employs a decentralized governance structure, with individual projects retaining autonomy over their leadership, decision-making, and development processes. This community-driven approach allows for rapid innovation while seeking community consensus around ecosystem-wide decisions and standards.

Consider, for example, a physicist studying particle collisions at the Large Hadron Collider (LHC) to investigate questions about the interactions of matter and the fundamental forces of Nature and search for evidence of new physics. They might use a deployment of the Institute for Research and Innovation in Software for High Energy Physics [7] (IRIS-HEP)'s Elastic Analysis Facility—a cloud-based platform that provides shared computing environments for high energy physics (HEP) researchers to process and analyze massive datasets, share code and results, and collaborate with colleagues globally. Then, to analyze collision data, the physicist might rely on tools from Scikit-HEP (see Figure 1), a SP-OSE sub-ecosystem specifically designed for Pythonic high energy physics analysis [71]. Scikit-HEP intentionally builds on a *stack* of packages, including foundational scientific Python libraries like NumPy for numerical computing [89, 28], SciPy for general scientific computing algorithms [88], Matplotlib for data visualization [33], and Pandas for data manipulation [49]. From this core stack, Scikit-HEP builds interoperable packages that collectively address more domain-specific computing operations. Physics driven data patterns and reduction choices are addressed through Awkward Array for handling complex hierarchical data structures [62] and boost-histogram and Hist for advanced multidimensional histogramming [72, 87]. Successively more HEP specific operations additionally use Uproot for I/O with the HEP-specific ROOT binary data format [61], Vector for physical and Lorentz vector transformations and operations [86], and Particle [45] for accessing particle properties from the Particle Data Group database [53].

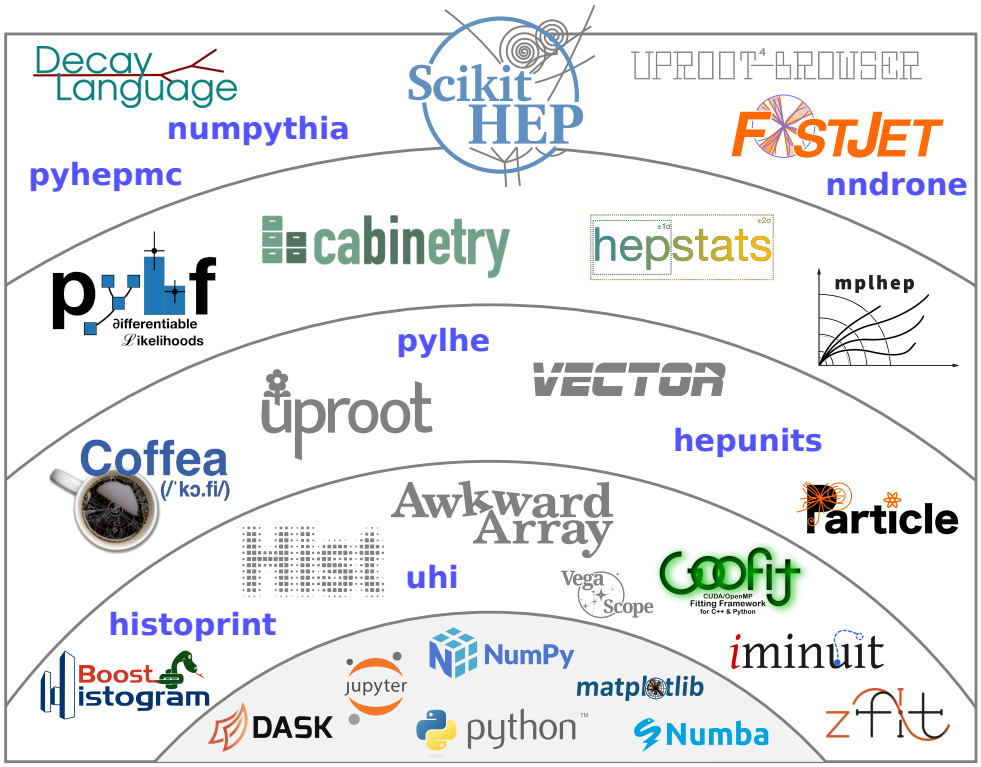



FIGURE 1. The components of the Python-based analysis ecosystem for HEP that form the Scikit-HEP domain stack. The bottom ring consists of the foundational computational libraries in the Scientific Python ecosystem. The successive rings contain tools that are more specifically designed for particle physics analysis and applications; while the first HEP-specific layer is broadly used across the field of particle physics, each layer becomes more operation- or experiment-specific.

This example demonstrates the concept of a *domain stack* in the SP-OSE, where libraries are organized in layers, with foundational packages at the base and increasingly specialized, domain-specific packages built on top. The foundational libraries provide essential functionality needed across many scientific domains, while upper-level packages extend this foundation to address the specific needs of researchers in particular fields. While the previous example focused on a physicist using the Scikit-HEP stack, a similar structure is found in other areas of science and engineering supported by the NSF. For instance, a geographer or urban scientist working with spatial data might rely on the PySAL ecosystem, which builds on foundational libraries such as NumPy, SciPy, and Pandas, and then adds specialized packages for spatial analysis, geostatistics, and visualization tailored to geographic research. There are many such domain stacks in the SP-OSE, including Astropy (astronomy), NIPY (neuroimaging), scverse (single-cell biology), and Biopython (bioinformatics), each layering domain-specific capabilities atop shared core libraries. Within a domain stack, each package typically has its own developer community,




FIGURE 2. Homepage of the Scientific Python Project showcasing its mission to coordinate and support the ecosystem through community-driven efforts, resources like lecture notes, development guides, and SPECs, as well as events such as developer summits.

communication channels, website, roadmap, documentation, engineering and packaging solutions, and governance structure. While this loosely coupled development model offers flexibility and fosters innovation, it also leads to redundancies and fragmentation across the SP-OSE. Such fragmentation poses significant challenges for sustainability, strategic direction, compatibility, user experience, collaboration, efficiency, performance, and deployment.

*Scientific Python Project (SPP).* To address these challenges, the PIs founded the **Scientific Python Project (SPP)** [73], the managing organization that steers the development of the SP-OSE, and the essential infrastructure needed to keep the ecosystem running (see Figure 2). SPP's mission is to better coordinate the SP-OSE and support its community of contributors and maintainers. Through the SPP, the PIs recruited a governing team of community leaders and managers, established central community spaces, brought developers together for cross-project Developer Summits, updated and maintained classic community-owned learning resources, aggregated and disseminated best development practices, and built and supported common maintainer tooling. SPP has grown rapidly,

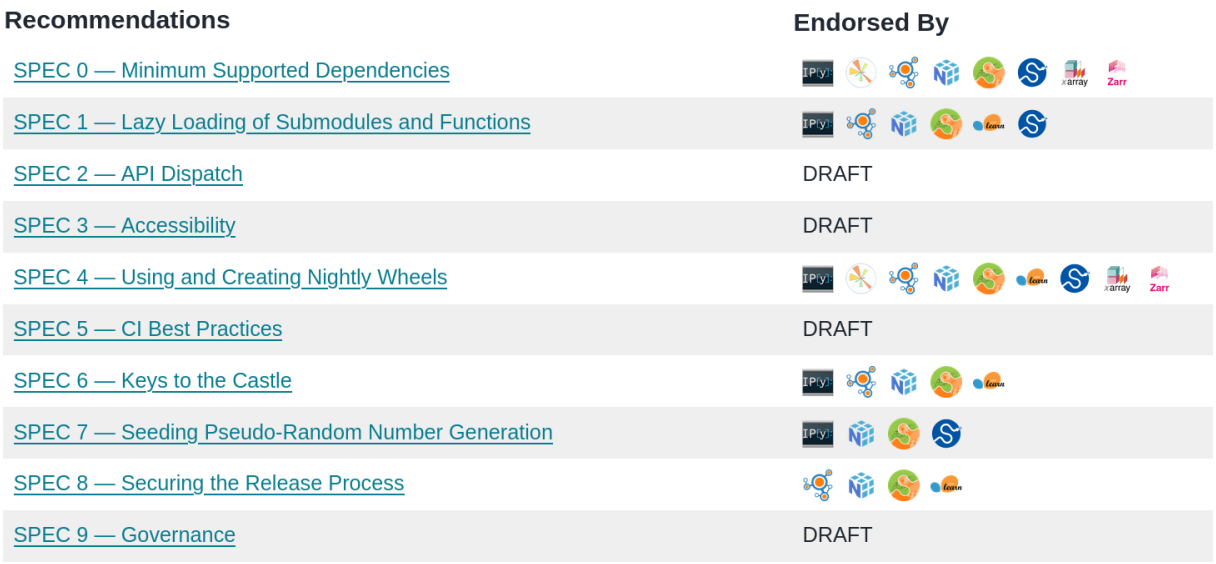



FIGURE 3. The growing list of SPECs and the SPEC Core Project endorsements. SPECs provide recommendations for projects in the scientific Python ecosystem. All community members and ecosystem projects are welcome to participate in the SPEC process.

with 80 team members and 356 followers for our GitHub organization, 29 affiliated Python libraries in our nightly wheel-hosting infrastructure, 332 subscribers to our YouTube channel, 477 members on our Discourse forum, and 694 members on our Discord server. We are holding our third annual Scientific Python Developer Summit this May.

All software is licensed under the modified BSD-3 license and educational material under Creative Commons licenses like CC0 or CC-BY. We follow an open and consensus-oriented governance model that encourages broad participation in activities, discussions, design, and decision-making processes, conducted transparently on open channels like GitHub. Membership is open to all, established through active collaboration, and requires adhering to the code of conduct. Decisions are made through community discussions on public forums following a consensus-seeking approach, with sensitive topics (e.g., security issues and conduct violations) addressed privately when necessary.

We created the **Scientific Python Ecosystem Coordination (SPEC)** process [76] to establish cross-project technical specifications, highlight shared resources, and document best practices (see Figure 3). The SPEC Steering Committee, comprising 17 volunteers from various projects in the ecosystem, oversees the SPEC process with input from the larger community and in consultation with the maintainers of the SPEC Core Projects.

The SPEC Core Projects are foundational projects that are depended upon by many other projects, and often provide basic data structures, drawing primitives, implementations of fundamental algorithms, or are metapackages that represent a particular scientific field. Due to their central position in the ecosystem, the policies, practices, and tooling used by the Core Projects are widely seen by the ecosystem and impact many other

projects. They follow open and collaborative practices, including having a public version control system and bug tracker, code of conduct, contributors' guide, code review process, public roadmap, documented governance system, enhancement proposal process, and regular releases. They are developed openly by their respective communities, with contributors from multiple institutions. They commonly use the 3-clause BSD license: a highly permissive free and open source software license, imposing minimal restrictions on use and re-distribution.

The growing list of SPEC Core Projects (in alphabetic order) currently includes:

- **ipython** (16.4k GH stars, 4.5k GH forks) [59], an interactive environment offering enhanced code execution and exploration, and the primary kernel for Jupyter;
- **matplotlib** (21k GH stars, 7.8k GH forks), a visualization library for the creation of high-quality plots for scientific analysis and communication;
- **NetworkX** (15.6k GH stars, 3.3k GH forks) [27], a library for the creation, manipulation, and analysis of complex networks or graphs;
- **NumPy** (29.3k GH stars, 10.7k GH forks), the fundamental array data structure used throughout the ecosystem;
- **pandas** (45.1k GH stars, 18.4k GH forks), a library for data manipulation and analysis, offering essential tools for tasks such as data cleaning, exploration, transformation, and statistical analysis;
- **PySAL** (1.4k GH stars, 307 GH forks) [79], a domain stack and Python metapackage for geospatial data science, supporting spatial analysis tasks such as detecting spatial clusters, modeling spatial relationships, and performing exploratory spatio-temporal data analysis;
- **scikit-hep** (165 GH stars, 35 GH forks) [85], a domain stack and a Python ecosystem tailored for high energy physics research, providing tools for handling particle physics datasets, histogramming, fitting models, and interfacing with HEP-specific libraries like ROOT;
- **scikit-image** (6.2k GH stars, 2.3k GH forks), an image processing library, offering tools for tasks such as image filtering, segmentation, and feature extraction [90];
- **scikit-learn** (61.7k GH stars, 25.8k GH forks) [58], a machine-learning library, offering tools for classification, regression, clustering, dimensionality reduction, etc.;
- **SciPy** (13.5k GH stars, 5.3k GH forks) [88], a collection of scientific computing routines, with algorithms for interpolation, linear algebra, optimization, signal processing, and more;
- **xarray** (3.8k GH stars, 1.1k GH forks), a library that implements labeled, multi-dimensional arrays;
- **zarr-python** (1.7k GH stars, 324 GH forks) [51], a library for efficient storage and manipulation of large, compressed, and chunked multi-dimensional arrays.

While GitHub stars are not a definitive measure of impact, it is notable that nine of the SPEC Core Projects rank among the top 0.036% of all repositories on GitHub by stars [38]. The maintainers of each of the SPEC Core Projects has agreed to become involved with the SPEC process; however, many more projects small and large participate and even more adopt the various SPECs for their own projects.

In addition to these projects, we also develop and maintain a growing collection of **SPP Tools and Services**—overseen by the Tools and Services Steering Committee committee—that support projects across the ecosystem, including those implementing the SPECs. The SPP currently supports approximately fifteen tools, ranging from release management utilities to community organization aids, as well as two core services: a discussion forum and web analytics. The expanding suite of developer tools includes:

- **cookie** (339 GH stars, 56 GH forks) [80], a template generator for new Python projects based on the Scientific Python Developer Guide. It ensures that new repositories adhere to best practices in development, testing, and deployment.
- **repo-review** (74 GH stars, 6 GH forks) [83], a framework for evaluating repositories against predefined guidelines from the Scientific Python Developer Guide. By identifying configuration issues and enforcing compliance with best practices, it helps maintain high standards across the ecosystem.
- **lazy-loader** (165 GH stars, 22 GH forks) [81], a library that implements lazy loading for Python submodules and functions, reducing import times and memory usage by loading components only when accessed.
- **MeeseeksDev** (36 GH stars, 22 GH forks) [9], a GitHub bot that automates routine project management tasks such as labeling issues, merging pull requests, backporting security fixes, and enforcing repository policies, thereby streamlining contributor workflows.
- **pytest-doctestplus** (104 GH stars, 40 GH forks) [82], a plugin for the pytest framework that extends doctest capabilities to support advanced testing of documentation files.
- **spin** (109 GH stars, 21 GH forks) [84], a developer tool designed to simplify common build and development tasks for scientific Python libraries. It abstracts complex command-line operations into user-friendly commands, reducing human error and ensuring consistent workflows across projects.
- **sync-teams-action** (4 GH stars, 2 GH forks) [19], a GitHub Action for managing teams and team membership within GitHub organizations, helping to automate and standardize access control.

Through these and other community initiatives, the SPP fosters interoperability, reduces maintenance burden, and provides a more cohesive experience for users.

3. **National, Societal, and Economic Impact.** The SP-OSE has far-reaching impacts on scientific research, education and workforce development, economic growth and innovation, and national security.

*Scientific Research.* The SP-OSE underpins cutting-edge research across a vast spectrum of scientific disciplines, from systems biology to materials science, and from astrophysics to climate modeling [44]. This ecosystem supports endeavors ranging from large international collaborations like CERN's particle physics experiments to critical national resources such as the Rubin Observatory [2, 23, 22], as well as countless individual research projects in universities, national laboratories, and research centers worldwide [39].

For example, in high energy physics, the ATLAS and CMS LHC experiments rely on scientific Python libraries for all stages of data analysis, including for the upcoming High-Luminosity LHC upgrade [37, 7, 1, 13, 74]. The Belle II experiment utilizes Scikit-HEP's vector and particle packages for precision measurements of CP violation in B-meson decays, critical for testing the Standard Model [15]. KM3NeT, the deep-sea neutrino telescope in the Mediterranean, employs Scikit-HEP tools for data analysis, simulation, and software development [70, 29]. Similarly, the LHCb Collaboration recently performed its first analysis entirely in Python, leveraging standard scientific Python packages alongside Scikit-HEP tools such as boost-histogram, iminuit, Particle, and uproot [14].

In geospatial science, foundational scientific Python and domain-specific PySAL packages have seen widespread adoption [69]. Specialized tools for spatial analysis—including geopandas, geoplot, and geosnap—build on these core projects. Moreover, the PySAL domain-stack is used by researchers from a wide array of topics across many disciplines including engineering [24], chemistry [77], physics [36], biology [55], neuroscience [8], epidemiology [32] and many others.

*Education and Workforce Development.* The ecosystem plays a crucial role in education, with Python increasingly adopted as the primary language taught at universities [57, 91, 47, 5]. At UC Berkeley, for instance, the adoption of SP-OSE has been foundational to the success of its Data Science education initiative, which includes the first new college in 50 years and the largest undergraduate major on campus [17]. Core courses such as Data 8: Foundations of Data Science—the university's largest single course—and Data 100: Principles and Techniques of Data Science—the largest upper-division course—rely heavily on the SP-OSE and the Jupyter-OSE to deliver high-impact, scalable instruction.

More than 50 universities and colleges across the United States have adopted components of Berkeley's teaching stack, with some institutions fully integrating the Data 8 curriculum as a foundational course in data science education [17]. Others have adapted individual elements to suit their disciplinary needs, embedding SP-OSE tools into courses across diverse fields. Through partnerships with initiatives like NSF's CloudBank project, Berkeley has supported community colleges, California State Universities (CSUs), and other UC campuses in adopting scalable cloud-based infrastructure and open-source educational resources [16].

Open-source workflows built on SP-OSE tools have become foundational in courses ranging from biology and physics to earth science and engineering [21]. In some majors, the SP-OSE serves as the standard computational toolkit, equipping students with transferable skills early in their academic journeys. In others, it supports advanced applications such as biosciences data analysis and earthquake monitoring, giving students authentic experiences with real-world scientific workflows.

*Economic Growth and Innovation.* The SP-OSE provides the backbone of the data science and machine learning revolution [30, 65, 34]. It underpins the majority of machine learning pipelines, enables risk analysis [31], supports medical image analysis and personalized medicine [10, 35, 46], and optimizes manufacturing processes [20, 42]. In the financial sector, algorithmic trading now accounts for over 60% of U.S. stock trades [68], with firms

like Citadel Securities and Two Sigma relying on libraries such as pandas, NumPy, and scikit-learn to process petabytes of market data daily [54, 64, 75, 67].

*National Security.* The SP-OSE is used to inform critical government decisions, to control expensive infrastructure, and to access sensitive health and scientific data—including operating on Department of Energy (DOE) clusters [66]. It enables sophisticated intelligence analysis, extracting actionable insights from vast datasets [11]. It also powers cutting-edge defense technologies, from complex simulations to target recognition algorithms, enhancing military capabilities and readiness [18].

4. **Targeted Classes of Vulnerabilities.** The SP-OSE faces several critical classes of vulnerabilities that, if exploited, could have profound consequences for scientific integrity, national security, and industry. While we describe these vulnerabilities separately for clarity, it is important to recognize that, in real-world attacks, they are often exploited in combination, creating complex and multifaceted threats. This interconnected nature of security risks underscores the need for a holistic approach to safeguarding the SP-OSE, one that addresses all classes of vulnerabilities simultaneously—a task that requires substantial work beyond what volunteer efforts alone can achieve. Below, we describe each class of vulnerability, and emphasize specific weaknesses and exploits we'd like to mitigate.

4.1. *Supply Chain Vulnerability (SCV).* The SP-OSE, as an extensive network of independent, interconnected libraries, is particularly susceptible to supply chain attacks. An attacker who successfully compromises a widely used dependency can insert malicious code that propagates throughout the ecosystem, impacting a vast array of downstream projects and users. This risk is highlighted by the recent Ultralytics supply-chain attack, which demonstrates the potential for widespread impact through compromised dependencies [43].

*(SCV1) Insufficient Code Signing Practices.* Many scientific Python projects lack cryptographic signing of software releases, exposing the ecosystem to dependency tampering and unauthorized modifications. Attackers exploit this gap to inject malicious code through methods like typosquatting or compromising outdated packages, as demonstrated by recent supply-chain incidents. We will develop community standards to implement cryptographic verification across dependencies, and provide tools to integrate it seamlessly into development workflows. By ensuring package authenticity, this work will strengthen trust in scientific software critical to national research infrastructure and the global economy.

*(SCV2) Fragmented Vulnerability Tracking.* The SP-OSE lacks coordinated vulnerability tracking, allowing attackers to exploit outdated transitive dependencies in projects like geospatial models or medical imaging pipelines. This gap enables attacks where malicious code persists in dependency chains for months or years, as seen in the 2024 Ultralytics breach where known vulnerabilities remained unpatched in 60% of downstream projects. We will establish ecosystem-wide protocols for vulnerability disclosure and, through automated dependency graph analysis, prioritize critical fixes. By closing this gap, we will protect sensitive research workflows in national laboratories and hospitals alike, while enabling safer collaboration across global open-source teams.

4.2. *Access Controls Vulnerability (ACV).* Attackers increasingly target open-source software projects using credential hijacking, where maintainer accounts are compromised through phishing attacks or password reuse, enabling malicious actors to commit code that appears legitimate. A compromised account with broad access could be used to insert malicious code into core libraries, affecting countless downstream projects and users across scientific domains, government research, and industry applications.

*(ACV1) Poor Authentication Requirements and Auditing.* Privileged accounts in the SP-OSE often lack multi-factor authentication (MFA), enabling attackers to hijack credentials through phishing or password reuse and gain unauthorized access. Furthermore, the absence of automated access audits allows such breaches to persist. We will establish community standards requiring MFA adoption for all project administrators and implement systems to continuously review permissions across projects. Strengthening these controls will protect sensitive data in healthcare and national security applications while fostering trust in open-source tools critical to scientific advancement.

*(ACV2) Coarse-grained Access Control.* The SP-OSE relies on coarse-grained permission models that grant excessive privileges to contributors, creating opportunities for accidental or malicious code modifications. Further, overprivileged automation tokens with broad write access increase the risk of backdoor injection during automated builds and deployments. We will collaborate with the community to implement and adopt role-based access controls, ensuring that contributors only have the permissions necessary to perform their allocated tasks. By reducing overprivileged access, this work will safeguard sensitive research data while maintaining the collaborative ethos of open-source development.

*(ACV3) Stale Maintainer Permissions.* A significant vulnerability arises in the SP-OSE when maintainers retain access rights long after they have ceased active contribution, resulting in dormant accounts that expand the project's attack surface. Attackers can exploit these inactive or unmonitored accounts to gain unauthorized entry and introduce malicious changes. We will establish protocols for regular review and revocation of inactive contributor permissions, ensuring that only actively engaged and verified maintainers retain elevated access. By reducing the number of stale or unnecessary privileged accounts, this work will significantly decrease the risk of account takeovers and unauthorized code changes, thereby strengthening the overall security and trustworthiness of the SP-OSE.

4.3. *Insider Attacks Vulnerability (IAV).* Social engineering attacks increasingly target maintainers experiencing burnout. Malicious actors exploit reduced vigilance due to fatigue and offer much needed "assistance", bypassing standard code review processes and introducing subtle backdoors. The ecosystem's approach, which encourages contributions from a global community of (mostly self-selecting) volunteers, expands the potential attack surface. This risk is compounded by a maintainer onboarding model that elevates privileges to active contributors without comprehensive background checks or identity verification. In this environment, sleeper agents—long-term contributors who gradually introduce vulnerabilities over time—pose a particularly insidious threat. These vulnerabilities have been starkly highlighted by recent high-profile attacks, such as the XZ Utils incident [26, 3], a multi-year effort—likely originating from a nation-state actor—that

underscores the community's vulnerability to sophisticated, patient attackers willing to establish long-term presence before executing their attacks.

*(IAV1) Poor Contributor Vetting.* The SP-OSE's open contribution model lacks systematic vetting of contributors, enabling attackers to gradually introduce vulnerabilities as trusted participants over months or years. This socio-technical vulnerability allows sleeper agents and those who socially engineered accounts to bypass code review processes. We will develop a reputation system analyzing contributors' ecosystem-wide activity to identify anomalous behavior while preserving the openness essential to collaborative development. By mitigating insider threats, this work will protect sensitive data in healthcare and national security applications while maintaining an open contribution model.

*(IAV2) Review Complexity and Overload.* The scale and interdependence of core libraries make it difficult for maintainers to fully assess risks, enabling attackers to hide malicious changes in complex codebases. Moreover, maintainers of widely used libraries often face overwhelming review workloads, creating opportunities for attackers to insert subtle, malicious code which passes unnoticed through rushed, fatigued inspections. We will develop automated systems to analyze contribution patterns and flag anomalies in code submissions, prioritizing high-risk changes for expert scrutiny. By augmenting human oversight with scalable screening tools, we will safeguard critical infrastructure in academic research and industrial systems while maintaining the open collaboration vital to scientific progress.

5. **Development Plan.** To address critical vulnerabilities and significantly improve the safety, security, and privacy of the SP-OSE, we will leverage the SPEC process. In collaboration with the SPP Community we will develop, and iteratively refine, ecosystem-wide security standards and best practices, building on and improving existing efforts as needed. Where necessary, we will also develop tools and services to help the community implement security SPECs effectively, paying careful attention to technical considerations such as the adoption of memory-safe languages and the generation of software bills of materials (SBOMs).

5.1. *Year 1 Milestones.* In the first year, our focus will be on SPEC-driven standardization, tool prototyping, and targeted deployment of security standards and best practices. We will start by operationalizing two existing security SPECs:

- SPEC 6 — Keys to the Castle [78], which provides principles and practical guidance for documenting, distributing, and managing access to restricted project resources and secrets, including privilege minimization, regular permission review, secure password storage, and the use of multi-factor authentication.
- SPEC 8 — Securing the Release Process [48], which establishes standards and procedures for safeguarding the software release pipeline using the Supply-chain Levels for Software Artifacts (SLSA) framework. SPEC 8 mandates SLSA Level 2 compliance for core projects, including cryptographic signing of packages, tamper-proof build processes, and generation of signed provenance attestations via GitHub Actions workflows. It integrates OpenID Connect (OIDC) for CI/CD pipeline authentication, replacing long-lived secrets with short-lived tokens, and provides templates for

generating in-toto attestations to ensure verifiable traceability from source code to distributed artifacts.

In parallel, we will refine existing SPECs and draft new ones to address gaps in vulnerability management and insider threat detection. Tooling improvements will be designed for seamless integration with the SPP infrastructure, supporting reproducible builds and automated policy enforcement. Feedback and lessons learned from pilot projects will inform further refinements, with focused attention on high-impact packages such as NumPy, SciPy, and scikit-learn to ensure ecosystem-wide applicability.

*Implementation Framework Year 1 Milestones*

- **Security Summit.** Host the First Scientific Python Security Summit with maintainers from core projects (NumPy, SciPy, pandas).
- **Public Dashboard.** Publish a public dashboard tracking SPEC adoption and implemenation information.

*Supply Chain Vulnerability Mitigation Year 1 Milestones*

- **SCV1—Code Signing.** This milestone implements SPEC 8's framework for securing the release pipeline through cryptographic signing and provenance verification. We will operationalize package integrity standards by deploying cryptographic signing workflows for scikit-image, NetworkX, and all SPP Tools and Services. Experience gained during implementation will inform revisions to SPEC 8, with an emphasis on simplicity and maintainer adoption.
- **SCV2—Vulnerability Management.** We will draft a SPEC for vulnerability management and incident response while developing a prototype federated CVE database with automated dependency graph analysis. This system will prioritize transitive risks in projects using C/C++ extensions.

*Access Control Vulnerability Mitigation Year 1 Milestones*

- **ACV1—MFA Adoption.** Implement multi-factor authentication requirements for all SPEC Core Project repository owners. This milestone codifies SPEC 6's recommendation for two-factor authentication.
- **ACV2—Role-based Access Control.** Deploy role-based access control (RBAC) prototypes in NetworkX and scikit-image, limiting CI/CD tokens to designated workflow files. The latter will be enforced through automated tools such as Zizmor and our repository audit tool, `repo-review`. This work operationalizes SPEC 6's privilege minimization guidance and will serve as a model for broader RBAC adoption.
- **ACV3—Access Audits.** Develop automated access audit systems, with protocols to revoke permissions after six months of inactivity. For example, we may improve on `sync-teams-action`, and roll it out to a wider set of projects, with `repo-review` checks in place to ensure that minimal permissions are given. This milestone automates SPEC 6's recommendation to regularly review permissions, reducing exposure from inactive accounts.

*Insider Attack Vulnerability Mitigation Year 1 Milestones*

- **IAV1—Reputation System.** We will research existing reputation systems and tools, and engage the community through the SPEC process to define requirements and use cases for a contributor reputation system. This system will, e.g., assess developer expertise, examine networks of collaboration, and incorporate a web of trust. We will then prototype the system, analyzing more than five years of contributor activity across ten core projects.
- **IAV2—Anomaly Detection.** We will review existing approaches and tools for anomaly detection, and work with the community through the SPEC process to clarify needs and desired features. Alpha versions, or configurations, of anomaly detection tools, that can also act as code review agents, will be released, with a focus on C, C++, and Cython codebases that use unsafe memory operations.

5.2. *Year 2 Milestones.* For the second year, we will focus on scaling and better integration of the tools, standards, and processes developed in Year 1 to achieve widespread adoption across the ecosystem. Our efforts will emphasize robust implementation of cryptographic signing, SBOM validation, automated vulnerability management, and advanced access controls in a broader set of projects. We will also refine and deploy reputation and anomaly detection systems at scale, ensuring that security best practices are embedded throughout the SP-OSE. By targeting both core and domain-specific projects, we aim to establish sustainable, community-driven security practices that can be maintained and evolved beyond the award period.

*Implementation Framework Year 2 Milestones*

- **Security Summit.** Host the Second Scientific Python Security Summit, focusing on domain-specific stacks (bioinformatics, geospatial) to drive adoption.
- **Public Dashboard.** Expand the public dashboard to include Critical Vulnerability & Exposure (CVE) tracking and provide dependency chain risk scores.

*Supply Chain Vulnerability Mitigation Year 2 Milestones*

- **SCV1—Code Signing and SBOM Validation.** Achieve 100% cryptographic signing adoption across SPEC Core Projects. Use the SPEC process to extend code signing practices for SBOM validation and integrate SBOM validation into PyPI and conda-forge pipelines.
- **SCV2—Vulnerability Management.** Provide automated notifications to downstream users regarding vulnerabilities. Integrate with existing dependency tracking tools to ensure comprehensive vulnerability detection and enable efficient patch prioritization for high-risk dependencies. Use the `meeseeksdev` backport bot to ensure that critical patches for updating dependencies are applied to patch releases. Automate CVE patch recommendations for nested dependencies in projects using legacy C or Fortran code.

*Access Control Vulnerability Mitigation Year 2 Milestones*

- **ACV1—MFA Adoption.** Extend MFA adoption to all contributors with write access in the top 20 SP-OSE projects.
- **ACV2—Role-based Access Control.** Scale RBAC implementation to 10 projects, including those with GPU or CUDA kernels requiring heightened privilege controls.
- **ACV3—Access Audits.** Implement quarterly automated access audits for all projects with more than 10 contributors, with the goal of reducing stale permissions by 70%.

*Insider Attack Vulnerability Mitigation Year 2 Milestones*

- **IAV1—Reputation System.** We will further develop and deploy the contributor reputation system across the entire ecosystem, enabling maintainers to flag contributors exhibiting anomalous or high-risk activity.
- **IAV2—Anomaly Detection.** We will refine and integrate anomaly detection tools into CI/CD workflows for projects utilizing unsafe memory operations, with the goal of reducing code review workloads. These tools will be deployed as automated checks for maintainers of high-risk packages, such as those handling cryptography or parallel computing. We will explore partnering with companies to train machine learning models for flagging suspicious patterns, such as obfuscated code or unexpected use of subprocesses, based on historical SP-OSE pull requests.

*Broader Impacts*

This initiative will systematically address critical vulnerabilities in the SP-OSE through the development and adoption of community-driven security standards, tools, and processes. By aligning our activities with the OpenSSF Open Source Software Security Mobilization Plan and CISA's Open Source Software Security Roadmap, we will ensure that the scientific Python ecosystem meets the highest standards of software security and resilience. Integration of frameworks such as OpenSSF's SLSA and federal cybersecurity mandates (e.g., NIST SSDF, CISA's Secure Software Attestation) into the SPEC process will establish replicable security models for open-source ecosystems worldwide.

By addressing these vulnerabilities, the project will significantly enhance the reliability, integrity, and trustworthiness of scientific software, which underpins research, education, and innovation across numerous scientific and engineering disciplines. The resulting improvements will help safeguard national security and critical infrastructure, as the SP-OSE is widely used in government, industry, and research settings. This work will also foster best practices in open-source development and vulnerability management, serving as a model for other software communities and contributing to improved economic competitiveness, resilience, and technological leadership for the United States.

Increased partnerships between academia, industry, government, and non-profit organizations will be facilitated through the collaborative nature of the project, promoting knowledge transfer and shared responsibility for open-source security. Ultimately, this initiative will not only secure the SP-OSE, but will also contribute to a more robust, vibrant

infrastructure for research and education, enabling trustworthy scientific and technological advancements that benefit society as a whole.

6. **Evaluation Plan.** To assess the efficacy of our work in enhancing the safety, security, and privacy of the SP-OSE, we will implement a comprehensive evaluation and accountability strategy:

*SPP Security Working Group Oversight.* We will establish a SPP Security Working Group by recruiting a diverse group of developers and maintainers from across the ecosystem to provide ongoing technical guidance and oversight of our security initiatives. This group will hold monthly meetings to evaluate progress, discuss challenges, and suggest directions to explore, ensuring that our efforts remain aligned with community needs and best practices. Based on what we learn in these meetings, we will develop sustainable strategies for the long-term implementation and maintenance of security measures beyond the grant period.

In short, the SPP Security Working Group provides technical guidance through focused developer feedback, as well as long term contiguity for our efforts.

*SPEC Process.* The SPEC Process is designed to ensure community input in the writing of and support for the implementation of proposed security initiatives. The SPEC Steering Committee ensures community ownership of decision-making on SPECs, fostering broad acceptance across the ecosystem. Endorsement from SPEC Core Projects projects provides a clear indication of the acceptance of our security initiatives within the core ecosystem. Project adoption demonstrates the reach and impact of our security measures beyond the core projects. The Tools and Services Steering Committee ensures we are creating common resources that address needs according to community standards, procedures, and practices. We aim to have each SPEC's security measures adopted by three projects within its first year, and seven projects by the end of its second. Additionally, we aim to have each SPEC *endorsed* by two core projects within its first year, and five by the end of its second. To make our efforts more visible, and to ensure broad endorsement of our SPECs, we will foster increased participation by various domain stacks in the SPEC process; their participation will be a clear signal that the work is gaining traction.

In short, the SPEC process provides general community accountability and transparent insight into the reach and impact of our security measures across the SP-OSE.

*External Advisory Board.* We will form an external advisory board with representatives from diverse SP-OSE end-user organizations. This board will include members from domain-specific projects (e.g., IRIS-HEP and Vera Rubin Observatory), industry (e.g., Google and NVIDIA), national computing facilities (e.g., San Diego Supercomputer Center and TrustedCI), and open-source foundations (e.g., PSF and OpenSSF), providing an authoratitive perspective on the impact of our work, rooted in day-to-day experience.

We will conduct quarterly virtual meetings with this board to ensure that our work aligns with stakeholder needs and expectations, allowing for timely adjustments to be made to our approach. Through these interactions, we will collect and analyze feedback to measure the perceived value and real-world impact of our implemented security measures.

In short, the External Advisory Board provides high-level strategic feedback on our work.

7. **Conclusion.** The SP-OSE stands at a critical juncture: as core infrastructure of national importance, it faces unprecedented security challenges that demand urgent attention. The proposed initiative is a coordinated, ecosystem-wide effort to comprehensively address critical safety, security, and privacy vulnerabilities in the SP-OSE. This ambitious undertaking demands not only technical expertise but also the ability to mobilize and unite a broad coalition of developer communities, spanning multiple scientific domains and industry sectors. So, while the work that needs to be done is challenging, it is of vital importance: it will strengthen the reliability and security of foundational scientific software, safeguarding national infrastructure and advancing best practices in open-source development, while fostering collaboration across academia, industry, government, and non-profits. Ultimately, these efforts will create a more resilient computational foundation for research, education, and innovation.

**PI Millman**, Executive Director of UC Berkeley's Open Source Program Office, brings 20+ years of leadership in open-source security and scientific computing. As cofounder of NumFOCUS [56] and former NumPy/SciPy release manager (2007–2009), he established governance models for foundational SP-OSE projects. He was an early contributor to scikit-learn and cofounder of the Neuroimaging in Python (NIPY) project, advancing open and reproducible research in both machine learning and neuroimaging domains. His cybersecurity expertise stems from roles as UC Berkeley's Neuroscience Institute Director of Computing (designing secure infrastructure for sensitive data) and service on the Campus Information Security and Privacy Committee (2006–2010). Millman is a maintainer and the release manager of NetworkX, scikit-image, pygraphviz, numpydoc, and many other packages.

**Co-PI van der Walt**, Research Scientist at the Berkeley Institute for Data Science, combines deep technical knowledge with ecosystem-wide coordination. A founding developer of scikit-image and coauthor of *Elegant SciPy*, he serves on the NumPy Steering Committee and co-chairs the Python Software Foundation's Scientific Working Group. His contributions span key SP-OSE projects including NumPy, SciPy, and NetworkX, with a focus on API design and community-driven governance. As NumFOCUS Emeritus Director, he bridges academic research and industrial adoption of SP-OSE tools.

**Project Manager Whitaker**, Executive Director of the Berkeley Institute for Data Science, brings expertise in building welcoming and collaborative open-source communities. Founder of *The Turing Way*—a best-practices handbook for ethical, reproducible data science—she previously led AI governance initiatives at the Alan Turing Institute. Her work at BIDS focuses on interdisciplinary collaboration, aligning with this project's need to harmonize security protocols across scientific domains.

The team's proven ability to coordinate cross-project development through the SPP ensures robust execution of this initiative. By leveraging their technical authority, institutional partnerships, and community trust, they will establish a secure foundation for global scientific computing.

## References


[1] Sam Albin, Garhan Attebury, Kenneth Bloom, Brian Bockelman, Carl Lundstedt, Oksana Shadura, and John Thiltges. "Coffea-Casa: Building composable analysis facilities for the HL-LHC". In: *EPJ Web Conf.* 295 (2024), p. 07009. DOI: 10.1051/epjconf/202429507009. arXiv: 2312.11485 [cs.DC].

[2] Francesco Andreuzzi. *Improving throughput of CERN's multi-language distributed online data processing platform*. Tech. rep. 2021.

[3] Andy Andy Greenberg and Matt Burgess. "The Mystery of 'Jia Tan,' the XZ Backdoor Mastermind". In: *WIRED* (2024).

[4] Astropy Collaboration, T. P. Robitaille, E. J. Tollerud, P. Greenfield, M. Droettboom, E. Bray, T. Aldcroft, M. Davis, A. Ginsburg, A. M. Price-Whelan, W. E. Kerzendorf, A. Conley, N. Crighton, K. Barbary, D. Muna, H. Ferguson, F. Grollier, M. M. Parikh, P. H. Nair, H. M. Unther, C. Deil, J. Woillez, S. Conseil, R. Kramer, J. E. H. Turner, L. Singer, R. Fox, B. A. Weaver, V. Zabalza, Z. I. Edwards, K. Azalee Bostroem, D. J. Burke, A. R. Casey, S. M. Crawford, N. Dencheva, J. Ely, T. Jenness, K. Labrie, P. L. Lim, F. Pierfederici, A. Pontzen, A. Ptak, B. Refsdal, M. Servillat, and O. Streicher. "Astropy: A community Python package for astronomy". In: *Astronomy & Astrophysics* 558, A33 (Oct. 2013), A33. DOI: 10.1051/0004-6361/201322068.

[5] Kaan Bati. "Integration of Python into Science Teacher Education, Developing Computational Problem Solving and Using Information and Communication Technologies Competencies of Pre-service Science Teachers". In: *Informatics in Education-An International Journal* 21.2 (2022), pp. 235–251.

[6] Moritz Beyreuther, Robert Barsch, Lion Krischer, Tobias Megies, Yannik Behr, and Joachim Wassermann. "ObsPy: A Python Toolbox for Seismology". In: *Seismological Research Letters* 81.3 (2010), pp. 530–533. DOI: 10.1785/gssrl.81.3.530.

[7] Brian Bockelman et al. "IRIS-HEP Strategic Plan for the Next Phase of Software Upgrades for HL-LHC Physics". In: (Feb. 2023). arXiv: 2302.01317 [hep-ex].

[8] Jonathan B. Burt, Murat Demirtas, William J. Eckner, Nicholas M. Navejar, Joseph L. Ji, William J. Martin, Alberto Bernacchia, Alan Anticevic, and John D. Murray. "Hierarchy of Transcriptomic Specialization Across Human Cortex Captured by Structural Neuroimaging Topography". In: *Nature Neuroscience* 21.9 (2018), pp. 1251–1260. DOI: 10.1038/s41593-018-0195-0. URL: https://www.nature.com/articles/s41593-018-0195-0.

[9] Matthias Bussonnier and contributors. *MeeseeksDev: Development deployment of MeeseeksBox (use at your own risk)*. https://github.com/scientific-python/MeeseeksDev. Version as of April 2025. 2017. URL: https://github.com/scientific-python/MeeseeksDev.

[10] Ravishankar Chityala and Sridevi Pudipeddi. *Image processing and acquisition using Python*. Chapman and Hall/CRC, 2020.

[11] Daniel R Coats. "The AIM Initiative: A Strategy for Augmenting Intelligence using Machines". In: *Office of the Director of National Intelligence* (2019).

[12] Astropy Collaboration et al. "The Astropy Project: sustaining and growing a community-oriented open-source project and the latest major release (v5. 0) of the core package". In: *Astrophysical Journal* 935.2 (2022), p. 167.

[13] ATLAS Collaboration. *ATLAS Software and Computing HL-LHC Roadmap*. Tech. rep. Geneva: CERN, 2022. URL: https://cds.cern.ch/record/2802918.

[14] LHCb Collaboration. "Measurement of prompt charged-particle production in $pp$ collisions at $\sqrt{s} = 13\,\text{TeV}$". In: *Journal of High Energy Physics* 2022.1 (2022), p. 166. DOI: 10.1007/JHEP01(2022)166. URL: https://link.springer.com/article/10.1007/JHEP01(2022)166.

[15] The Belle II Collaboration. "The Belle II Core Software (basf2): Simulation, Reconstruction, and Analysis Framework". In: *EPJ Web of Conferences*. Vol. 295. EDP Sciences, 2024, p. 05018. DOI: 10.1051/epjconf/202429505018. URL: https://doi.org/10.1051/epjconf/202429505018.

[16] College of Computing, Data Science, and Society (CDSS). *Expanding Access to Data Science Education Through Partnerships and Open-Source Resources*. 2025. URL: https://cdss.berkeley.edu/dsus/nwdse.

[17] College of Computing, Data Science, and Society (CDSS). *UC Berkeley Data Science Major Now Largest on Campus*. 2025. URL: https://cdss.berkeley.edu/dsus/news/data-science-major-now-largest-campus.

[18] Joao PA Dantas, Samara R Silva, Vitor CF Gomes, Andre N Costa, Adrisson R Samersla, Diego Geraldo, Marcos ROA Maximo, and Takashi Yoneyama. "AsaPy: A Python Library for Aerospace Simulation Analysis". In: *Proceedings of the 38th ACM SIGSIM Conference on Principles of Advanced Discrete Simulation*. 2024, pp. 15–24.

[19] Scientific Python Developers. *sync-teams-action: Synchronize team membership and permissions for GitHub organizations*. https://github.com/scientific-python/sync-teams-action. Version as of April 2025. 2023. URL: https://github.com/scientific-python/sync-teams-action.

[20] T Kalavathi Devi, EB Priyanka, and P Sakthivel. "Paper quality enhancement and model prediction using machine learning techniques". In: *Results in Engineering* 17 (2023), p. 100950.

[21] Eric Van Dusen, Anthony Suen, Alan Liang, and Amal Bhatnagar. "Accelerating the Advancement of Data Science Education". In: *Proceedings of the 18th Python in Science Conference (SciPy 2019)*. SciPy, 2019, pp. 52–59. DOI: 10.25080/Majora-7ddc1dd1-000. URL: https://doi.org/10.25080/Majora-7ddc1dd1-000.

[22] Frédéric Dux. "lightcurver: A Python Pipeline for Precise Photometry of Multiple-Epoch Wide-Field Images". In: *arXiv preprint arXiv:2410.13625* (2024).

[23] V Daniel Elvira, Steven Gottlieb, Oliver Gutsche, and Benjamin Nachman. *The Future of High Energy Physics Software and Computing (V2. 1)*. Tech. rep. Fermi National Accelerator Laboratory (FNAL), Batavia, IL (United States), 2022.

[24] Y. Fan, X. Zhu, B. She, W. Guo, and T. Guo. "Network-Constrained Spatio-Temporal Clustering Analysis of Traffic Collisions in Jianghan District of Wuhan, China". In: *PLOS ONE* 13.4 (2018), e0195093. DOI: 10.1371/journal.pone.0195093. URL: https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0195093.

[25] GitHub. *Octoverse 2024: The state of open source and AI*. GitHub. 2024. URL: https://github.blog/news-insights/octoverse/octoverse-2024/ (visited on 11/21/2024).

[26] Kaspersky Global Research & Analysis Team. "XZ backdoor story Part 2: Assessing the Y, and How, of the XZ Utils incident (social engineering)". In: *Securelist* (2024).

[27] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. "Exploring Network Structure, Dynamics, and Function using NetworkX". In: *Proceedings of the 7th Python in Science Conference*. Ed. by Gaël Varoquaux, Travis Vaught, and K. Jarrod Millman. Pasadena, CA USA, 2008, pp. 11–15.

[28] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. "Array programming with NumPy". In: *Nature* 585.7825 (Sept. 2020), pp. 357–362. DOI: 10.1038/s41586-020-2649-2. URL: https://doi.org/10.1038/s41586-020-2649-2.

[29] Lukas Hennig. "Tau Neutrino Identification with Graph Neural Networks in KM3NeT/ORCA". MA thesis. Erlangen, Germany: Friedrich-Alexander-Universität Erlangen-Nürnberg, 2023. URL: https://ecap.nat.fau.de/wp-content/uploads/2023/06/2023-06_LukasHennig_MSc_ORCA_TauID_GNNs.pdf.

[30] Tony Hey, Keith Butler, Sam Jackson, and Jeyarajan Thiyagalingam. "Machine learning and big scientific data". In: *Philosophical Transactions of the Royal Society A* 378.2166 (2020), p. 20190054.

[31] Yves Hilpisch. *Python for Algorithmic Trading*. O'Reilly Media, 2020.

[32] C. Hughes, V. S. Naik, R. Sengupta, and D. Saxena. "Geovisualization for Cluster Detection of Hepatitis A & E Outbreaks in Ahmedabad, Gujarat, India". In: *Proceedings of the Third ACM SIGSPATIAL International Workshop on the Use of GIS in Public Health*. ACM, 2014, pp. 39–44. DOI: 10.1145/2684108.2684117. URL: https://dl.acm.org/doi/10.1145/2684108.2684117.

[33] John D Hunter. "Matplotlib: A 2D graphics environment". In: *Computing in Science & Engineering* 9.3 (2007), pp. 90–95. URL: https://doi.org/10.1109/MCSE.2007.55.

[34] Laura Igual and Santi Seguí. "Introduction to data science". In: *Introduction to Data Science: A Python Approach to Concepts, Techniques and Applications*. Springer, 2024, pp. 1–4.

[35] Sultan Imangaliyev, Bart Keijser, Wim Crielaard, and Evgeni Tsivtsivadze. "Personalized microbial network inference via co-regularized spectral clustering". In: *Methods* 83 (2015), pp. 28–35.

[36] Anna Jakubska-Busse, Mariusz W. Janowicz, Leszek Ochnio, and Julian M. A. Ashbourn. "Pickover Biomorphs and Non-Standard Complex Numbers". In: *Chaos, Solitons & Fractals* 113 (2018), pp. 46–52. DOI: 10.1016/j.chaos.2018.05.020. URL: https://www.sciencedirect.com/science/article/pii/S0960077918302091.

[37] Philipp Alexander Jung. "b-hive: a modern start-to-finish flavor tagging framework". In: *PoS* LHCP2024 (2025), p. 275. DOI: 10.22323/1.478.0275.

[38] K0kubun. *Gitstar Ranking: GitHub Star Ranking for Users, Organizations, and Repositories*. Available at https://gitstar-ranking.com. 2025. URL: https://gitstar-ranking.com.

[39] Alok Kamatar, Mansi Sakarvadia, Valerie Hayot-Sasson, Kyle Chard, and Ian Foster. "Lazy python dependency management in large-scale systems". In: *2023 IEEE 19th International Conference on e-Science (e-Science)*. IEEE. 2023, pp. 1–10.

[40] Thomas Kluyver, Benjamin Ragan-Kelley, Fernando Pérez, Brian Granger, Matthias Bussonnier, Jonathan Frederic, Kyle Kelley, Jessica Hamrick, Jason Grout, Sylvain Corlay, Paul Ivanov, Damián Avila, Safia Abdalla, and Carol Willing. "Jupyter Notebooks – a publishing format for reproducible computational workflows". In: *Positioning and Power in Academic Publishing: Players, Agents and Agendas*. Ed. by F. Loizides and B. Schmidt. IOS Press. 2016, pp. 87 –90.

[41] Lion Krischer, Tobias Megies, Robert Barsch, Moritz Beyreuther, Thomas Lecocq, Corentin Caudron, and Joachim Wassermann. "ObsPy: a bridge for seismology into the scientific Python ecosystem". In: *Computational Science & Discovery* 8.1 (2015), p. 014003. DOI: 10.1088/1749-4699/8/1/014003.

[42] Daniel J Laky, Daniel Casas-Orozco, Mesfin Abdi, Xin Feng, Erin Wood, Gintaras V Reklaitis, and Zoltan K Nagy. "Using PharmaPy with Jupyter Notebook to teach digital design in pharmaceutical manufacturing". In: *Computer applications in engineering education* 31.6 (2023), pp. 1662–1677.

[43] Larson, Seth. *Supply-chain attack analysis: Ultralytics*. 2024.

[44] Johnny Wei-Bing Lin, Hannah Aizenman, Erin Manette Cartas Espinel, Kim Gunnerson, and Joanne Liu. *An introduction to Python programming for scientists and engineers*. Cambridge University Press, 2022.

[45] University of Liverpool and Princeton University. *scikit-hep/particle: Release 0.25.3*. Version v0.25.3. 2025. DOI: 10.5281/zenodo.15132783. URL: https://zenodo.org/record/15132783.

[46] Vijay Mane, Mohinee Jadhav, and Ishita Kadam. "Human Disease Prediction Using Machine Learning". In: *2024 International Conference on Emerging Smart Computing and Informatics (ESCI)*. IEEE. 2024, pp. 1–6.

[47] Anatoly B Mantusov, Zamira B Dorzhinova, et al. "Python language in the development of computational competencies in humanities education". In: *European Proceedings of Social and Behavioural Sciences* (2021).

[48] Matthew Feickert, Pamphile Roy, Juanita Gomez, Seth Larson, Lars Grüter, Jarrod Millman. *SPEC 8 — Securing the Release Process*. URL: https://scientific-python.org/specs/spec-0008/.

[49] Wes McKinney. "Data Structures for Statistical Computing in Python". In: *Proceedings of the 9th Python in Science Conference*. Ed. by Stéfan van der Walt and Jarrod Millman. 2010, pp. 51–56.

[50] Tobias Megies, Moritz Beyreuther, Robert Barsch, Lion Krischer, and Joachim Wassermann. "ObsPy – What can it do for data centers and observatories?" In: *Annals Of Geophysics* 54.1 (2011), pp. 47–58. DOI: 10.4401/ag-4838.

[51] Alistair Miles, Joe Hamman, Dimitri Papadopoulos Orfanos, David Stansby, M Bussonnier, Josh Moore, Davis Bennett, Norman Rzepka, Deepak Cherian, Sanket Verma, James Bourbeau, Andrew Fulton, Ryan Abernathey, Gregory Lee, Hannes Spitz, Mads R. B. Kristensen, Max Jones, Zain Patel, Saransh Chopra, Matthew Rocklin, Martin Durant, Juan Nunez-Iglesias, and Raphael Dussin. *zarr-developers/zarr-python: v2.18.7*. Version v2.18.7. 2025. DOI: 10.5281/zenodo.15180703. URL: https://zenodo.org/record/15180703.

[52] K Jarrod Millman and Michael Aivazis. "Python for scientists and engineers". In: *Computing in science & engineering* 13.2 (2011), pp. 9–12.

[53] S. Navas et al. "Review of Particle Physics". In: *Phys. Rev. D* 110 (3 2024), p. 030001. DOI: 10.1103/PhysRevD.110.030001. URL: https://link.aps.org/doi/10.1103/PhysRevD.110.030001.

[54] PyQuant News. *A short interview with a front office quant at Citadel*. LinkedIn post by PyQuant News, July 16, 2024. 2024. URL: https://www.linkedin.com/posts/pyquant-news_a-short-interview-with-a-front-office-quant-activity-7219008634192621568-Gco4.

[55] Javad Noorbakhsh, Saman Farahmand, Mehdi Soltanieh-ha, Sridevi Namburi, Kamran Zarringhalam, and Jason Chuang. "Pan-Cancer Classifications of Tumor Histological Images Using Deep Learning". In: *bioRxiv* (2019). DOI: 10.1101/715656. URL: https://www.biorxiv.org/content/10.1101/715656v1.

[56] *NumFOCUS*. URL: https://numfocus.org.

[57] Randy C Paffenroth and Xiangnan Kong. "Python in Data Science Research and Education." In: *SciPy*. 2015, pp. 164–170.

[58] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. "Scikit-learn: Machine Learning in Python". In: *Journal of Machine Learning Research* 12.Oct (2011), pp. 2825–2830. URL: http://www.jmlr.org/papers/volume12/pedregosa11a/pedregosa11a.pdf.

[59] Fernando Pérez and Brian E Granger. "IPython: a system for interactive scientific computing". In: *Computing in Science & Engineering* 9.3 (2007), pp. 21–29. URL: https://doi.org/10.1109/MCSE.2007.53.

[60] Fernando Perez, Brian E Granger, and John D Hunter. "Python: an ecosystem for scientific computing". In: *Computing in Science & Engineering* 13.2 (2011), pp. 13–21.

[61] Jim Pivarski, Pratyush Das, Chris Burr, Dmitri Smirnov, Matthew Feickert, Tamas Gal, Luke Kreczko, Nicholas Smith, Noah Biederbeck, Oksana Shadura, Mason Proffitt, benkrikler, Hans Dembinski, Henry Schreiner, Jonas Rembser, Marcel R., Chao Gu, Edoardo Rodrigues, JMSchoeffmann, Jonas Rübenach, Lukas Koch, Michele Peresano, Niclas Eich, and Ruggero Turra. *scikit-hep/uproot3: 3.14.4*. Version 3.14.4. 2021. DOI: 10.5281/zenodo.4537826. URL: https://zenodo.org/record/4537826.

[62] Jim Pivarski, Ianna Osborne, Ioana Ifrim, Henry Schreiner, Angus Hollands, Anish Biswas, Pratyush Das, Santam Roy Choudhury, Nicholas Smith, and Manasvi Goyal. *Awkward Array*. Oct. 2018. DOI: 10.5281/zenodo.4341376.

[63] A. M. Price-Whelan et al. "The Astropy Project: Building an Open-science Project and Status of the v2.0 Core Package". In: *The Astronomical Journal* 156 (Sept. 2018), p. 123.

[64] Interview Query. *Citadel LLC Data Scientist Interview Questions + Guide in 2025*. 2025. URL: https://www.interviewquery.com/interview-guides/citadel-data-scientist.

[65] Sebastian Raschka, Joshua Patterson, and Corey Nolet. "Machine learning in python: Main developments and technology trends in data science, machine learning, and artificial intelligence". In: *Information* 11.4 (2020), p. 193.

[66] Samantha Ravich and Thomas Rosenbaum. *SEAB Artificial Intelligence and Machine Learning Working Group*. Tech. rep. U.S. Department of Energy, 2020.

[67] Jeff Reback. *The Future of Pandas*. 2020. URL: https://www.twosigma.com/articles/the-future-of-pandas/.

[68] Allied Market Research. *Algorithmic Trading Market Size, Share, Industry Analysis - 2032*. 2023. URL: https://www.alliedmarketresearch.com/algorithmic-trading-market-A08567.

[69] Sergio J. Rey and Elijah Knaap. "The PySAL Ecosystem: Philosophy and Implementation". In: *Center for Geospatial Sciences, University of California, Riverside* (2021). URL: https://cogs.sdsu.edu/publication/rey-2021-py-sal-ecosystem/.

[70] Eduardo Rodrigues. *HEP Data Analysis in Python Ecosystem and Community Efforts*. Manchester HEP Seminar, All@Home. 2020. URL: https://indico.cern.ch/event/961730/attachments/2157295/3638863/2020-12-04_ManchesterSeminar.pdf.

[71] Eduardo Rodrigues et al. "The Scikit HEP Project – overview and prospects". In: *EPJ Web Conf.* 245 (2020). Ed. by C. Doglioni, D. Kim, G. A. Stewart, L. Silvestris, P. Jackson, and W. Kamleh, p. 06028. DOI: 10.1051/epjconf/202024506028. arXiv: 2007.03577 [physics.comp-ph].

[72] Henry Schreiner, Hans Dembinski, Shuo Liu, and Jim Pivarski. "Boost-histogram: High-Performance Histograms as Objects". In: *Proceedings of the 19th Python in Science Conference*. Ed. by Meghann Agarwal, Chris Calloway, Dillon Niederhut, and David Shupe. 2020, pp. 63 –69. DOI: 10.25080/Majora-342d178e-009.

[73] Scientific Python Team. *Scientific Python Project Homepage*. URL: https://scientific-python.org/.

[74] David Richard Shope. "Software Upgrades for the High-Luminosity LHC". In: *PoS* LHCP2024 (2025), p. 193. DOI: 10.22323/1.478.0193.

[75] Two Sigma. *pandas at a Crossroads: The Past, Present, and Future with Jeff Reback*. 2023. URL: https://www.twosigma.com/articles/pandas-at-a-crossroads-the-past-present-and-future-with-jeff-reback/.

[76] SPEC Steering Committee. *Scientific Python Ecosystem Coordination*. URL: https://scientific-python.org/specs.

[77] L. Spiridon and D. D. Minh. "Hamiltonian Monte Carlo with Constrained Molecular Dynamics as Gibbs Sampling". In: *Journal of Chemical Theory and Computation* 13.10 (2017), pp. 4649–4659. DOI: 10.1021/acs.jctc.7b00625. URL: https://pubs.acs.org/doi/10.1021/acs.jctc.7b00625.

[78] Stéfan van der Walt, Jarrod Millman, Pamphile Roy, Lars Grüter. *SPEC 6 — Keys to the Castle*. URL: https://scientific-python.org/specs/spec-0006/.

[79] PySAL Development Team. *PySAL: Python Spatial Analysis Library Meta-Package*. Available at https://github.com/pysal/pysal. 2025. URL: https://github.com/pysal/pysal.

[80] Scientific Python Development Team. *Cookie: A Copier/Cookiecutter Template for Scientific Python Projects*. Available at https://github.com/scientific-python/cookie. 2025. URL: https://github.com/scientific-python/cookie.

[81] Scientific Python Development Team. *lazy-loader: Populate library namespace lazily*. Available at https://github.com/scientific-python/lazy-loader. 2025. URL: https://github.com/scientific-python/lazy-loader.

[82] Scientific Python Development Team. *pytest-doctestplus: Advanced Doctest Support for Pytest*. Available at https://github.com/scientific-python/pytest-doctestplus. 2025. URL: https://github.com/scientific-python/pytest-doctestplus.

[83] Scientific Python Development Team. *repo-review: Framework for Repository Compliance Checks*. Available at https://github.com/scientific-python/repo-review. 2025. URL: https://github.com/scientific-python/repo-review.

[84] Scientific Python Development Team. *Spin: Developer Tool for Scientific Python Libraries*. Available at https://github.com/scientific-python/spin. 2025. URL: https://github.com/scientific-python/spin.

[85] Scikit-HEP Development Team. *Scikit-HEP: Metapackage for Particle Physics Data Analysis*. Available at https://github.com/scikit-hep/scikit-hep. 2025. URL: https://github.com/scikit-hep/scikit-hep.

[86] Princeton University and University of Delhi. *Vector: A Python library for 2D, 3D, and Lorentz vectors*. Version v0.10.0. 2022. DOI: 10.5281/zenodo.7054478. URL: https://zenodo.org/record/7054478.

[87] Princeton University, TU Dortmund, The University of Manchester, CERN, UCL UCL-ARC, Amazon, University of Wisconsin-Madison, and @sezane. *scikit-hep/boost-histogram: Version 1.5.2*. Version 1.5.2. 2025. DOI: 10.5281/zenodo.15098538. URL: https://zenodo.org/record/15098538.

[88] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, S. J. van der Walt, M. Brett, J. Wilson, K. J. Millman, N. Mayorov, A. R. J. Nelson, E. Jones, R. Kern, E. Larson, C. J. Carey, İ. Polat, Y. Feng, E. W. Moore, J. VanderPlas, D. Laxalde, J. Perktold, R. Cimrman, I. Henriksen, E. A. Quintero, C. R. Harris, A. M. Archibald, A. H. Ribeiro, F. Pedregosa, P. van Mulbregt, and SciPy 1.0 Contributors. "SciPy 1.0—Fundamental Algorithms for Scientific Computing in Python". In: *Nature Methods* 17 (2020), pp. 261–272.

[89] Stéfan van der Walt, S Chris Colbert, and Gaël Varoquaux. "The NumPy array: a structure for efficient numerical computation". In: *Computing in Science & Engineering* 13.2 (2011), pp. 22–30. URL: https://doi.org/10.1109/MCSE.2011.37.

[90] Stéfan van der Walt, Johannes L Schönberger, Juan Nunez-Iglesias, François Boulogne, Joshua D Warner, Neil Yager, Emmanuelle Gouillart, Tony Yu, and the scikit-image contributors. "scikit-image: image processing in Python". In: *PeerJ* 2 (2014), e453. URL: https://doi.org/10.7717/peerj.453.

[91] Xiaofei Zhang, Jiajin Huang, Yang Yang, Xiang He, Ruohao Liu, and Ning Zhong. "Applying python in brain science education". In: *2019 International Joint Conference on Information, Media and Engineering (IJCIME)*. IEEE. 2019, pp. 396–400.

NSF Safe-OSE: Securing the Scientific Python Ecosystem

## Data Management and Sharing Plan

*Types of Data Collected.* This project will primarily collect and generate the following types of data:

1. Security vulnerability reports and assessments within the Scientific Python Open Source Ecosystem (SP-OSE).
2. Community feedback on proposed security standards and best practices.
3. Technical documentation for security mechanisms and protocols.
4. Usage statistics and metrics related to the adoption of security measures.
5. Anonymized data on security incidents and their resolutions.
6. Educational materials on open-source security practices.

*Data Collection and Storage.* All data will be collected through secure channels, ensuring the privacy and confidentiality of contributors. Sensitive information, such as unreleased vulnerability reports, will be encrypted and stored in secure repositories with restricted access. Non-sensitive data will be stored in public repositories on platforms like GitHub, following best practices for open-source project management.

*Data Sharing and Access.* We are committed to open science and will make all non-sensitive data publicly available. This includes:

- Technical documentation and educational materials, which will be published on the project website and GitHub repositories.
- Anonymized and aggregated security metrics, which will be shared through regular public reports.
- Community feedback and discussions, which will be archived in public forums or mailing lists.

Access to sensitive data, such as ongoing vulnerability investigations, will be strictly controlled. Only authorized team members and relevant stakeholders will have access to this information, governed by clear access policies and secure authentication mechanisms.

*Protection of Sensitive Data.* To protect sensitive data:

- We will implement end-to-end encryption for all communications regarding unreleased vulnerabilities.
- Access to sensitive repositories will require multi-factor authentication.
- We will use anonymization techniques to remove identifiable information from security incident reports before public release.

*Data Retention and Archiving.* Non-sensitive project data will be retained indefinitely on public platforms to ensure long-term accessibility for the community. Sensitive data related to resolved security issues will be securely archived for a period of five years, after which it will be thoroughly reviewed for potential public release or secure deletion.

*Licensing and Attribution.* All public data, documentation, and educational materials produced by this project will be released under the Creative Commons Attribution 4.0 International License (CC BY 4.0). This license allows for maximum reuse and adaptation while ensuring proper attribution. Any code produced as part of the project will be released under the BSD 3-Clause License, consistent with many Scientific Python projects.

*Ethical Considerations and IRB Approval.* While we do not anticipate collecting personal data that would require IRB approval, we will continuously assess our data collection practices. If any activities arise that involve human subjects research, we will seek IRB approval before proceeding. We are committed to ethical data practices and will prioritize the privacy and security of all participants and contributors.

*Community Engagement and Data Management.* We will actively engage with the Scientific Python community to ensure our data management practices align with community expectations and needs. Regular updates on our data management practices will be provided through the Scientific Python Ecosystem Coordination (SPEC) process. We will establish a clear process for community members to request access to or correction of any data related to them.

By implementing this comprehensive Data Management and Sharing Plan, we aim to balance open science principles with the need for security and privacy in the context of our project to secure the Scientific Python Open Source Ecosystem.

## Facilities

PI Millman and van der Walt, through the Berkeley Institute for Data Science, have access to conference rooms, offices, and a large open work space in UC Berkeley's Sutardja Dai Hall. The conference rooms of the UC Berkeley Statistics Department are also available for meetings.

NSF Safe-OSE: Securing the Scientific Python Ecosystem

**Project Personnel, Collaborators, and Partner Organizations**

(1) K. Jarrod Millman; UC Berkeley; PI
(2) Stéfan J. van der Walt; UC Berkeley; Co-PI
(3) Kirstie J. Whitaker; UC Berkeley; Project Manager
(4) Gordon Watts; University of Washington; Letter of Collaboration
(5) Frank Wuerthwein; UC San Diego; Letter of Collaboration
(6) Seth Larson; Python Software Foundation; Letter of Collaboration
(7) Dustin Ingram; Google; Letter of Collaboration
(8) Pradeep Gupta; NVIDIA; Letter of Collaboration